

Ejemplos de atención y conceptos básicos del Transformer

Ejemplos paso a paso para atención encoder-decoder, atención global, atención local, self-attention, positional encoding, RoPE, ALiBi y modelos Transformer

Resumen

Este documento explica, con ejemplos manuales y no con código, la ruta conceptual que conecta la atención encoder-decoder descrita por Weng, la atención aditiva de Bahdanau, la atención global y local de Luong, y el Transformer de *Attention Is All You Need*. Los ejemplos se construyen alrededor de tres situaciones concretas: la secuencia **The dog is running and then swimming in the lake**, el vector de entrada $x_2 = E(\text{runs}) + PE(2)$, y el sesgo ALiBi $[-3m, -2m, -m, 0]$. La intención es que el estudiante pueda leer cada matriz de atención como una pregunta lingüística: quién mira a quién, por qué lo mira, qué información recupera y cómo cambia la representación final del token.

Índice

1. Ruta general del documento	3
2. De oración a secuencia de tokens	3
3. Atención encoder-decoder y relación con Weng	4
4. Atención aditiva de Bahdanau: paso a paso	4
5. Atención global: ejemplo manual completo	5
5.1. Paso 1: definir la pregunta del token consultante	5
5.2. Paso 2: asignar scores globales	5
5.3. Paso 3: aplicar softmax	5
5.4. Paso 4: multiplicar por los valores	5
5.5. Ventaja y costo de la atención global	5
6. Atención local de Luong: ejemplo manual paso a paso	6
6.1. Paso 1: elegir el token consultante y la posición central	6
6.2. Paso 2: definir una ventana	6
6.3. Paso 3: crear una máscara local	6
6.4. Paso 4: scores locales y softmax local	6
6.5. Paso 5: interpretación	7
6.6. Comparación directa entre global y local	7
7. Self-attention y softmax	7
8. Q, K y V: lectura conceptual con la oración del perro	8

9. Codificación posicional clásica: $x_2 = E(\text{runs}) + PE(2)$	8
9.1. Relación con Word2Vec	9
10. RoPE con ejemplo manual	9
11. ALiBi con pendiente m: ejemplo manual	10
12. Multi-head attention con cabeceras diferenciadas	10
13. Complejidad $O(T^2)$ y división de secuencias	11
14. Bloque Transformer completo	11
15. Encoder, decoder y cross-attention	12
16. Máscara causal, decoder-only y encoder-only	12
17. Pérdida next-token	12
18. Decoding: temperatura, top-k, top-p, greedy y beam search	13
19. Entrenamiento y optimización	13
20. Preentrenamiento y fine-tuning	13
21. Resumen final	13

1. Ruta general del documento

La idea que une todos los temas es simple: un modelo de lenguaje necesita construir representaciones de tokens que no dependan solo del token aislado, sino también del contexto. En Word2Vec, un token como **runs** puede tener un vector estático porque aparece en contextos parecidos a **walks** o **jumps**. En un Transformer, ese vector inicial se transforma en una representación contextualizada. Por eso **runs** no debe significar exactamente lo mismo en **the dog runs fast** que en **the program runs slowly**.

La ruta conceptual es la siguiente. Primero, el seq2seq clásico comprime toda la oración de entrada en un vector de contexto fijo. Segundo, la atención encoder-decoder permite que el decoder consulte distintas partes de la entrada en cada paso. Tercero, Bahdanau introduce atención aditiva, donde el alineamiento se aprende con una red pequeña. Cuarto, Luong organiza variantes globales y locales: global mira toda la entrada; local restringe la atención a una ventana. Quinto, self-attention lleva la atención dentro de una misma secuencia. Sexto, el Transformer reemplaza recurrencia y convolución por multi-head attention, codificación posicional, conexiones residuales, LayerNorm y FFN.

Usaremos como oración principal:

The dog is running and then swimming in the lake.

Gramaticalmente es una sola oración compuesta o coordinada. Tiene un sujeto principal, **The dog**, y dos acciones coordinadas:

is running + and then swimming in the lake.

El segundo sujeto está implícito: el perro corre y luego nada en el lago. Para un Transformer, esta oración puede procesarse como una sola secuencia de tokens o dividirse artificialmente. La decisión depende de la tarea y del preprocesamiento, no de una obligación interna del Transformer.

2. De oración a secuencia de tokens

Una tokenización didáctica por palabras sería:

[The, dog, is, running, and, then, swimming, in, the, lake].

Asignamos posiciones desde cero:

Posición	0	1	2	3	4	5	6	7	8	9
Token	The	dog	is	running	and	then	swimming	in	the	lake

Un tokenizador real puede producir subpalabras o tokens con espacios incorporados. Por ejemplo, modelos tipo GPT pueden distinguir **dog** de **dog**, porque el espacio anterior puede formar parte del token. SentencePiece suele representar espacios con un marcador especial. El punto técnico es que el Transformer no recibe palabras como objetos lingüísticos; recibe identificadores enteros, y esos identificadores se convierten en vectores mediante una matriz de embeddings.

Si **running** se divide en subpalabras, la secuencia puede cambiar. Por ejemplo:

[The, dog, is, run,ning, and, then, swim,ming, in, the, lake].

Entonces la matriz de atención ya no relaciona exactamente palabras, sino tokens o subpalabras. Aun así, la lógica se mantiene: las piezas asociadas a **swimming** pueden atender a **dog** y **lake**.

3. Atención encoder-decoder y relación con Weng

En un seq2seq clásico, un encoder lee la entrada y produce un vector de contexto fijo. Si la entrada es **She is eating a green apple**, ese vector debe resumir sujeto, verbo, artículo, adjetivo y objeto. Luego el decoder usa ese vector para generar la salida. Este diseño funciona con frases cortas, pero se vuelve limitado con secuencias largas: todo debe caber en una sola representación.

La atención encoder-decoder cambia esta idea. En lugar de un único vector fijo, el decoder calcula un contexto dinámico en cada paso. Si está generando la traducción de **apple**, debería mirar más el estado del encoder asociado a **apple**. Si genera la traducción de **green**, debería mirar más **green**. Esta idea es la que Weng presenta como una distribución de importancia: el modelo aprende pesos que indican qué partes de la entrada son relevantes para producir una salida.

Formalmente, si el encoder produce estados $h_1, \dots, h_n$, el decoder construye:

$$c_t = \sum_{i=1}^n \alpha_{t,i} h_i,$$

donde $\alpha_{t,i}$ es el peso de atención desde el paso de salida t hacia la posición de entrada i . La suma de los pesos es 1:

$$\sum_i \alpha_{t,i} = 1.$$

Con la oración del perro, si un decoder estuviera traduciendo al español y en cierto paso quisiera generar **nadando**, el contexto dinámico debería mirar con fuerza a **swimming**. Pero también debería mirar a **dog**, porque el sujeto está implícito en la segunda acción, y a **lake**, porque ayuda a cerrar el significado de nadar.

4. Atención aditiva de Bahdanau: paso a paso

La atención aditiva de Bahdanau calcula un puntaje de alineamiento entre el estado previo del decoder y cada estado del encoder. Una forma didáctica de escribirlo es:

$$e_{t,i} = v_a^\top \tanh(W_s s_{t-1} + W_h h_i),$$
$$\alpha_{t,i} = \text{softmax}(e_{t,i}).$$

La palabra *aditiva* se entiende porque el estado del decoder y el estado del encoder se combinan dentro de una transformación aprendida antes de producir el score.

Supongamos que el decoder está generando **nadando**. La entrada tiene diez tokens. Asignamos puntajes manuales de alineamiento:

Token fuente	The	dog	is	running	and	then	swimming	in	the	lake
Score $e_{t,i}$	0.1	1.7	0.0	0.4	0.2	0.8	2.1	0.3	0.0	1.2

Después de aplicar softmax, una distribución aproximada puede ser:

Token fuente	The	dog	is	running	and	then	swimming	in	the	lake
Peso $\alpha_{t,i}$	.04	.20	.04	.06	.05	.10	.30	.06	.04	.14

La interpretación es lingüística y vectorial al mismo tiempo. El decoder atiende más a **swimming**, pero también incorpora **dog** y **lake**. En otras palabras, el contexto c_t no es solo la palabra alineada, sino una mezcla ponderada de información relevante.

5. Atención global: ejemplo manual completo

La atención global evalúa todas las posiciones de entrada. Es decir, para generar o contextualizar un token, el modelo no restringe la búsqueda a una ventana local. En la oración:

The dog is running and then swimming in the lake,

si el token consultante es **swimming**, una atención global compara **swimming** con las diez posiciones.

5.1. Paso 1: definir la pregunta del token consultante

La query de **swimming** puede interpretarse como: *necesito saber quién realiza la acción, qué acción estoy representando, qué marcador temporal me conecta con lo anterior y cuál es el lugar*. Por tanto, se espera que haya peso hacia **dog**, **then**, **swimming** y **lake**.

5.2. Paso 2: asignar scores globales

Usamos scores manuales para mostrar el mecanismo:

Token atendido	The	dog	is	running	and	then	swimming	in	the	lake
Score	-0.2	1.8	0.1	0.6	0.2	1.0	2.3	0.4	-0.1	1.6

Estos scores todavía no son probabilidades. Solo indican compatibilidad entre la query de **swimming** y las keys de todos los tokens.

5.3. Paso 3: aplicar softmax

Al aplicar softmax, obtenemos una distribución aproximada:

Token atendido	The	dog	is	running	and	then	swimming	in	the	lake
Peso	.03	.21	.04	.07	.04	.10	.35	.05	.03	.18

La fila completa suma 1. Una lectura posible es: **swimming** se atiende a sí mismo para conservar identidad, atiende a **dog** para recuperar sujeto y atiende a **lake** para recuperar lugar. También hay una atención moderada hacia **then**, porque marca secuencia temporal.

5.4. Paso 4: multiplicar por los values

Después, estos pesos se multiplican por los vectores V . Si V_{dog} codifica información de sujeto, $V_{swimming}$ codifica la acción y V_{lake} codifica lugar, entonces el nuevo vector de **swimming** contiene una mezcla de esos contenidos. Por eso decimos que la salida es un embedding contextualizado.

5.5. Ventaja y costo de la atención global

La ventaja es que no pierde relaciones lejanas. **swimming** puede mirar directamente a **dog**, aunque estén separados por varios tokens. El costo es que para una secuencia de longitud T , una cabecera construye una matriz $T \times T$. Con $T = 10$, hay 100 comparaciones por cabecera. Con $T = 1000$, hay un millón de comparaciones por cabecera.

6. Atención local de Luong: ejemplo manual paso a paso

La atención local restringe la mirada a una región de la entrada. La idea es aproximar la atención global, pero con menor costo y con una preferencia por zonas cercanas o alineadas. En Luong, la atención local puede estimar una posición central p_t y usar una ventana alrededor de ella.

6.1. Paso 1: elegir el token consultante y la posición central

Tomemos otra vez el token consultante `swimming`, ubicado en posición 6:

Posición	0	1	2	3	4	5	6	7	8	9
Token	The	dog	is	running	and	then	swimming	in	the	lake

Supongamos que la atención local predice una posición central:

$$p_t = 6.$$

Esto significa: el modelo cree que lo más relevante está alrededor de `swimming`.

6.2. Paso 2: definir una ventana

Si usamos una ventana de radio $D = 2$, se permiten posiciones desde $p_t - D$ hasta $p_t + D$:

$$[4, 5, 6, 7, 8].$$

Es decir:

$$[\text{and, then, swimming, in, the}].$$

El token `lake` queda fuera si solo usamos radio 2. Si queremos incluirlo, podemos usar radio 3:

$$[3, 4, 5, 6, 7, 8, 9].$$

6.3. Paso 3: crear una máscara local

Con radio 2, la máscara local para las diez posiciones sería:

Posición	0	1	2	3	4	5	6	7	8	9
Permitida	0	0	0	0	1	1	1	1	1	0

Las posiciones con 0 no entran al softmax. En la práctica reciben un valor muy negativo antes de softmax, por ejemplo $-\infty$ o un número como -10^9 .

6.4. Paso 4: scores locales y softmax local

Supongamos que los scores originales para todas las posiciones eran:

Token	The	dog	is	running	and	then	swimming	in	the	lake
Score	-0.2	1.8	0.1	0.6	0.2	1.0	2.3	0.4	-0.1	1.6

Después de aplicar la máscara local con radio 2, solo quedan:

Token local	and	then	swimming	in	the
Score local	0.2	1.0	2.3	0.4	-0.1

Al aplicar softmax solo sobre esa ventana, una distribución aproximada es:

Token local	and	then	swimming	in	the
Peso local	.07	.15	.57	.08	.05

6.5. Paso 5: interpretación

La atención local se concentra en **swimming** y algo en **then**. Esto representa bien el evento local, pero pierde **dog** y **lake** con radio 2. Con radio 3 sí incluiría **lake**, pero aún dejaría fuera **dog**. Esto muestra una tensión importante: atención local reduce costo, pero puede perder dependencias lejanas. En varias capas, un modelo puede recuperar parte de esa información indirectamente, pero la atención global la obtiene de forma directa.

6.6. Comparación directa entre global y local

Criterio	Atención global	Atención local
Alcance	Mira todas las posiciones de entrada.	Mira una ventana alrededor de una posición central.
Costo	Mayor, porque usa toda la matriz $T \times T$.	Menor si la ventana es pequeña.
Ejemplo con swimming	Puede atender a dog , then , swimming y lake .	Con radio 2 atiende a and , then , swimming , in , the .
Riesgo	Computacionalmente costosa en secuencias largas.	Puede perder relaciones lejanas como swimming → dog .

7. Self-attention y softmax

Self-attention ya no alinea una salida con una entrada, sino posiciones dentro de la misma secuencia. En nuestra oración, **running** puede atender a **dog**; **swimming** puede atender a **dog** y **lake**; **then** puede relacionar los dos eventos.

La operación central es:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

El producto $QK^\top$ produce puntajes. El escalado por $\sqrt{d_k}$ evita que los productos punto crezcan demasiado cuando la dimensión es alta. El softmax convierte cada fila en pesos que suman 1. La multiplicación por V produce el vector contextualizado.

Supongamos que para **swimming** se consideran cuatro tokens relevantes:

[**dog**, **running**, **swimming**, **lake**].

Asignamos scores:

$$[2, 1, 0, 8, 2, 4, 1, 7].$$

Después de softmax:

$$[0, 31, 0, 08, 0, 42, 0, 19].$$

La lectura es: el token conserva su identidad, pero recupera información del sujeto y del lugar.

8. Q, K y V: lectura conceptual con la oración del perro

Cada token produce tres vectores:

- Q , query: lo que el token busca.
- K , key: cómo el token puede ser encontrado.
- V , value: qué contenido aporta si recibe atención.

Para **swimming**, la query puede buscar sujeto, evento anterior y lugar. Las keys de **dog**, **then**, **swimming** y **lake** pueden ser compatibles. Los values contienen el contenido que se mezclará.

Token consultante	Token atendido	Motivo
running	dog	Recupera el sujeto de la primera acción.
swimming	dog	Recupera el sujeto implícito de la segunda acción.
swimming	lake	Recupera el lugar que completa la acción de nadar.
then	running, swimming	Marca la transición temporal entre dos eventos.

La matriz $QK^\top$ no es todavía la información final. Es una matriz de compatibilidad. El contenido final aparece cuando los pesos multiplican V .

9. Codificación posicional clásica: $x_2 = E(\text{runs}) + PE(2)$

La expresión:

$$x_2 = E(\text{runs}) + PE(2)$$

significa que el vector que entra al Transformer para **runs** combina significado léxico y posición.

Si el vocabulario tiene tamaño V y la dimensión del modelo es d_{model} , la matriz de embeddings es:

$$E \in \mathbb{R}^{V \times d_{model}}.$$

Si **runs** tiene ID 17, entonces:

$$E(\text{runs}) = E[17].$$

Con $d_{model} = 6$, un ejemplo manual es:

$$E(\text{runs}) = [0, 21, -0, 44, 0, 70, 0, 12, -0, 08, 0, 33].$$

El vector posicional de la posición 2 puede ser:

$$PE(2) = [0, 91, -0, 41, 0, 02, 0, 99, 0, 00, 1, 00].$$

Se suman componente a componente:

$$\begin{aligned}x_2 &= [0,21, -0,44, 0,70, 0,12, -0,08, 0,33] \\&\quad + [0,91, -0,41, 0,02, 0,99, 0,00, 1,00] \\&= [1,12, -0,85, 0,72, 1,11, -0,08, 1,33].\end{aligned}$$

Técnicamente, x_2 representa el token **runs** ubicado en la posición 2. Sin $PE(2)$, el Transformer tendría dificultad para distinguir **the dog runs** de **runs dog the**. La codificación posicional no reemplaza el significado del token; lo coloca dentro de la secuencia.

9.1. Relación con Word2Vec

Word2Vec aprende vectores estáticos para palabras. Dos palabras con contextos parecidos tienden a tener vectores cercanos. En un Transformer, la matriz de embeddings cumple un papel inicial parecido, pero no es el resultado final. Después de sumar posición y pasar por self-attention, el vector se vuelve contextualizado.

Por eso **runs** puede comenzar con el mismo embedding en:

the dog runs fast

y en:

the program runs slowly,

pero sus representaciones finales serán distintas. En el primer caso se asocia a movimiento físico; en el segundo, a ejecución de software.

10. RoPE con ejemplo manual

RoPE, Rotary Positional Embedding, no suma un vector posicional al embedding. En lugar de modificar directamente x , rota Q y K según la posición. La ventaja es que el producto punto entre Q y K queda sensible a distancias relativas.

Usemos un ejemplo 2D. Supongamos:

$$q_{\text{runs}} = [1, 0], \quad k_{\text{dog}} = [1, 0].$$

Asignemos ángulos didácticos: posición 1 rota 30 grados y posición 2 rota 60 grados. La matriz de rotación es:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

Entonces:

$$R(30^\circ)k_{\text{dog}} = [0,866, 0,500],$$

$$R(60^\circ)q_{\text{runs}} = [0,500, 0,866].$$

El producto punto es:

$$[0,500, 0,866] \cdot [0,866, 0,500] = 0,866.$$

Este valor refleja una diferencia de 30 grados entre las posiciones. Si **dog** estuviera más lejos, la diferencia angular sería distinta y el score cambiaría. Esa es la intuición de RoPE: la posición entra en la geometría del producto punto.

11. ALiBi con pendiente m : ejemplo manual

ALiBi, Attention with Linear Biases, no suma posición al embedding y no rota Q ni K . Agrega un sesgo lineal directamente a los logits de atención antes de softmax. Ese sesgo depende de la distancia y de una pendiente m .

Tomemos la secuencia:

Posición	Token
0	the
1	dog
2	runs
3	fast

Si el token actual es **fast** en posición 3, las distancias hacia atrás son:

Token atendido	Distancia	Sesgo ALiBi
the	3	$-3m$
dog	2	$-2m$
runs	1	$-m$
fast	0	0

Si $m = 0,5$:

$$[-3m, -2m, -m, 0] = [-1,5, -1,0, -0,5, 0].$$

Supongamos scores de contenido antes de ALiBi:

$$[0,3, 1,1, 1,5, 1,2].$$

Después de sumar ALiBi:

$$[0,3, 1,1, 1,5, 1,2] + [-1,5, -1,0, -0,5, 0] = [-1,2, 0,1, 1,0, 1,2].$$

Aplicando softmax, los pesos aproximados son:

$$[0,04, 0,15, 0,37, 0,45].$$

La penalización no prohíbe atender lejos. Solo exige que un token lejano tenga un score de contenido suficientemente fuerte para competir. En multi-head attention, cada cabecera puede usar una pendiente distinta: una cabecera con m grande favorece mucho lo cercano; otra con m pequeño permite dependencias más largas.

12. Multi-head attention con cabeceras diferenciadas

Multi-head attention ejecuta varias atenciones en paralelo. En lugar de una única matriz de atención, se calculan varias matrices, una por cabecera. Cada cabecera puede capturar una relación distinta. Evitamos pensar que las cabeceras vienen etiquetadas de fábrica; las etiquetas siguientes son una interpretación didáctica.

Cabecera	Patrón didáctico	Ejemplo
Cabecera sintáctica	sujeto-verbo	running → dog , swimming → dog
Cabecera locativa	acción-lugar	swimming → lake
Cabecera temporal	orden de eventos	then → running , then → swimming
Cabecera local	cercanía inmediata	swimming → then , swimming → in

Después de calcular las salidas parciales, se concatenan y se proyectan:

$$\text{MHA}(X) = \text{Concat}(O^{(1)}, O^{(2)}, \dots, O^{(h)})W^O.$$

La concatenación reúne perspectivas diferentes; W^O mezcla esas perspectivas en un vector final por token.

13. Complejidad $O(T^2)$ y división de secuencias

La matriz de atención de una cabecera tiene tamaño $T \times T$. Si la oración completa tiene 10 tokens:

$$T = 10 \quad \Rightarrow \quad T^2 = 100$$

comparaciones por cabecera.

Si se divide artificialmente:

The dog is running.

Then it is swimming in the lake.

una tokenización simple puede dar longitudes 4 y 7. El costo separado sería:

$$4^2 + 7^2 = 16 + 49 = 65.$$

Se reduce el costo, pero también cambia el contexto. En la secuencia completa, **swimming** puede atender directamente a **dog**. En la versión dividida, si solo se procesa la segunda oración, el modelo ve **it**, pero no necesariamente ve **dog**. Para mantener la referencia, ambas oraciones deben estar dentro de la misma ventana de contexto o debe existir algún mecanismo de memoria.

14. Bloque Transformer completo

La atención no trabaja sola. Un bloque Transformer típico combina multi-head attention, conexión residual, LayerNorm y FFN:

$$H = \text{LayerNorm}(X + \text{MHA}(X)),$$

$$Y = \text{LayerNorm}(H + \text{FFN}(H)).$$

La atención mezcla información entre tokens. La conexión residual conserva la señal original. LayerNorm estabiliza la escala. La FFN transforma cada token de forma no lineal después de incorporar contexto.

Para **swimming**, el recorrido es:

1. Comienza como embedding léxico más información posicional.
2. Self-attention lo conecta con **dog**, **then** y **lake**.
3. La conexión residual conserva parte de su identidad original.
4. LayerNorm estabiliza la representación.
5. FFN refina el vector para que represente **swimming** como segunda acción del perro en el lago.

15. Encoder, decoder y cross-attention

En un encoder, self-attention suele ser bidireccional. Cada token puede mirar todos los tokens de la entrada. Esto sirve para comprensión. Si el encoder procesa la oración completa, **running** puede mirar **dog**, **swimming** puede mirar **dog** y **lake**, y **then** puede conectar ambos eventos.

En un decoder autoregresivo, la atención es causal. El token actual no puede mirar el futuro. Si el prefijo es:

The dog is running and then,

el modelo debe predecir **swimming** sin verlo todavía.

En un encoder-decoder, aparece cross-attention. El decoder produce queries; el encoder aporta keys y values. Si el modelo traduce la oración al español, cuando el decoder genera **nadando**, sus queries pueden atender a las keys asociadas a **swimming**, **dog** y **lake**.

16. Máscara causal, decoder-only y encoder-only

La máscara causal bloquea posiciones futuras. Para:

[The, dog, is, running],

la posición de **is** puede mirar **The**, **dog** e **is**, pero no **running**. Las posiciones futuras reciben un valor muy negativo antes de softmax, lo que las deja con probabilidad casi cero.

Un modelo decoder-only usa esta máscara en todas sus capas y entrena con predicción del siguiente token. Es adecuado para generación. Un modelo encoder-only no usa máscara causal y es útil para clasificación, etiquetado o comprensión. Un encoder-decoder combina un encoder bidireccional con un decoder causal y cross-attention.

17. Pérdida next-token

El entrenamiento autoregresivo desplaza la secuencia. Con:

The dog is running and then swimming in the lake,

se crean pares como:

Prefijo	Token objetivo
The	dog
The dog	is
The dog is	running
The dog is running and then	swimming
The dog is running and then swimming in the	lake

Si el modelo asigna probabilidad 0.70 al token correcto, la pérdida local es:

$$-\log(0,70) \approx 0,357.$$

Si asigna 0.10:

$$-\log(0,10) \approx 2,303.$$

La pérdida castiga con más fuerza cuando el modelo asigna baja probabilidad al siguiente token correcto.

18. Decoding: temperatura, top-k, top-p, greedy y beam search

Durante generación, el modelo produce una distribución para el siguiente token. Si el prefijo es:

The dog is,

una distribución hipotética puede ser:

Token candidato	Probabilidad
running	0.55
swimming	0.20
walking	0.12
barking	0.08
otros	0.05

Greedy elige siempre el máximo. Beam search conserva varias hipótesis y compara secuencias completas. Temperatura baja concentra la distribución; temperatura alta aumenta diversidad. Top-k restringe a los k candidatos más probables. Top-p selecciona el conjunto mínimo cuya probabilidad acumulada alcanza p .

19. Entrenamiento y optimización

AdamW ajusta parámetros con weight decay desacoplado. Warmup sube gradualmente el learning rate al inicio. Cosine decay lo reduce después. Gradient clipping limita la norma del gradiente. Label smoothing evita que el modelo se vuelva excesivamente confiado.

Con la oración del perro, si el modelo aprende que después de **The dog is** suele venir **running**, pero también puede venir **swimming**, una distribución suavizada evita una certeza absoluta. Eso mejora calibración y reduce sobreajuste en corpus pequeños o repetitivos.

20. Preentrenamiento y fine-tuning

En preentrenamiento decoder-only, el modelo aprende next-token prediction sobre grandes corpus. Aprende patrones de sujeto-verbo, dependencias temporales, ubicaciones plausibles y continuidad discursiva. La oración **The dog is running and then swimming in the lake** resume varias señales: sujeto compartido, dos acciones, marcador temporal y complemento locativo.

En fine-tuning, el modelo se adapta a una tarea. Para traducción, aprende salidas como **El perro está corriendo y luego nadando en el lago**. Para clasificación, puede detectar que la oración describe actividad física de un animal. Para extracción de eventos, puede devolver dos eventos: correr y nadar.

21. Resumen final

Los ejemplos muestran que atención no significa mirar todo sin criterio. Significa construir una distribución de relevancia. La atención global permite que **swimming** mire directamente a **dog** y **lake**; la atención local reduce costo, pero puede perder relaciones lejanas si la ventana es pequeña. La suma $E(\text{runs}) + PE(2)$ combina significado inicial y posición. RoPE introduce posición relativa rotando Q y K . ALiBi agrega un sesgo como $[-3m, -2m, -m, 0]$ para penalizar distancia. Multi-head

attention usa varias cabeceras para capturar patrones sintácticos, temporales, locativos y locales. El Transformer integra estas piezas en un bloque entrenable que puede funcionar como encoder-only, decoder-only o encoder-decoder.

Referencias

- [1] Vaswani, A. et al. *Attention Is All You Need*. 2017.
- [2] Weng, L. *Attention? Attention!*. 2018.
- [3] Bahdanau, D., Cho, K., Bengio, Y. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2015.
- [4] Luong, M.-T., Pham, H., Manning, C. D. *Effective Approaches to Attention-based Neural Machine Translation*. 2015.
- [5] Su, J. et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021.
- [6] Press, O., Smith, N. A., Lewis, M. *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*. 2021.